

Calculus

April 14, 2026

1 Calculus

1.1 Contents

- Limits
- Limit of a Function
- One-Sided Limits
- Limits at Infinity
- Limit of a Sequence
- Limit Superior and Limit Inferior
- Derivatives (Single Variable)
- Derivative Notation
- Higher Order Derivatives
- Evaluation at a Point
- Dot Notation
- D Operator Notation
- Vector-Valued Derivative
- Partial Derivatives (Scalar-Valued)
- Partial Derivatives
- Directional Derivative
- Higher Order Partial Derivatives
- Gradient
- Laplacian
- Hessian
- Derivatives (Vector-Valued)
- Jacobian
- Curl
- Divergence
- Integration
- Indefinite Integral
- Definite Integral
- Multiple Integrals
- Domain Subscripts
- Line Integral and Contour Integral
- Convolution and Transforms
- Convolution
- Laplace Transform
- Fourier Transform

- Discrete Fourier Transform
- Fourier Series

1.2 Notes

- The notebooks are largely self-contained. If a symbol appears, an explanation will be present at some point in the same notebook — most often with a link to the cell where it is explained.
- **GitHub does a poor job of rendering these notebooks.** The online render is missing links, symbols are absent, and notations are badly formatted. Clone a local copy or download the notebook and open it locally, or refer to the PDFs in the PDFs/ folder.
- All code uses only Python's standard library.
- Calculus concepts are demonstrated using numerical methods (finite differences for derivatives, Riemann sums for integrals). These approximate results match analytical solutions to high precision.
- See the [Functions notebook](#) for background on function notation used throughout this notebook.

1.3 Importing Libraries

```
[1]: import math
import cmath
```

The following helper functions compute numerical approximations using the central difference method. They are reused throughout this notebook.

The `deriv` function approximates the derivative of a single-variable function at a point using the central difference formula:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

```
[2]: def deriv(f, x, h=1e-7):
    return (f(x + h) - f(x - h)) / (2 * h)

# verify: derivative of sin(x) at x = 0 is cos(0) = 1
deriv(math.sin, 0)
```

```
[2]: 0.9999999999999983
```

The `partial_deriv` function approximates the partial derivative of a multivariable function with respect to its i -th argument at a given point, using the central difference formula:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_i - h, \dots, x_n)}{2h}$$

Parameters: `f` (callable), `point` (sequence of float), `i` (zero-based index of variable), `h` (step size, default `1e-7`).

```
[3]: def partial_deriv(f, point, i, h=1e-7):
      x_fwd = list(point)
      x_bwd = list(point)
      x_fwd[i] += h
      x_bwd[i] -= h
      return (f(*x_fwd) - f(*x_bwd)) / (2 * h)

      # verify: d/dx(x * y) at (2, 3) = y = 3
      partial_deriv(lambda x, y: x * y, [2, 3], 0)
```

[3]: 2.9999999995311555

Limits

The principal notation for limits and the behavior of functions as variables approach specific values or grow without bound.

Limit of a Function

The limit of a function $f(x)$ as x approaches a value a is the value that $f(x)$ gets arbitrarily close to as x gets arbitrarily close to a . It is denoted by:

$$\lim_{x \rightarrow a} f(x)$$

If this limit equals some value L , the statement is written:

$$\lim_{x \rightarrow a} f(x) = L$$

```
[4]: # lim_{x -> 0} sin(x) / x = 1
f = lambda x: math.sin(x) / x
for x in [1.0, 0.1, 0.01, 0.001, 0.0001]:
    print(f'x = {x:<8} f(x) = {f(x):.10f}')
```

```
x = 1.0      f(x) = 0.8414709848
x = 0.1      f(x) = 0.9983341665
x = 0.01     f(x) = 0.9999833334
x = 0.001    f(x) = 0.9999998333
x = 0.0001   f(x) = 0.9999999983
```

One-Sided Limits

A one-sided limit considers the behavior of $f(x)$ as x approaches a from only one direction. The left-hand limit is denoted:

$$\lim_{x \rightarrow a^-} f(x) = \lim_{x \uparrow a} f(x)$$

The right-hand limit is denoted:

$$\lim_{x \rightarrow a^+} f(x) = \lim_{x \downarrow a} f(x)$$

The two-sided limit $\lim_{x \rightarrow a} f(x)$ exists if and only if both one-sided limits exist and are equal.

```
[5]: # f(x) = x / |x| has different one-sided limits at x = 0
f = lambda x: x / abs(x)
print('Approaching 0 from the right (+):')
for x in [1.0, 0.1, 0.01, 0.001]:
    print(f' x = {x:<8} f(x) = {f(x):+.1f}')
print('Approaching 0 from the left (-):')
for x in [-1.0, -0.1, -0.01, -0.001]:
    print(f' x = {x:<8} f(x) = {f(x):+.1f}')
```

Approaching 0 from the right (+):

x = 1.0	f(x) = +1.0
x = 0.1	f(x) = +1.0
x = 0.01	f(x) = +1.0
x = 0.001	f(x) = +1.0

Approaching 0 from the left (-):

x = -1.0	f(x) = -1.0
x = -0.1	f(x) = -1.0
x = -0.01	f(x) = -1.0
x = -0.001	f(x) = -1.0

Limits at Infinity

The limit of $f(x)$ as x increases or decreases without bound is written:

$$\lim_{x \rightarrow \infty} f(x) \quad \text{and} \quad \lim_{x \rightarrow -\infty} f(x)$$

```
[6]: # lim_{x -> inf} (1 + 1/x)^x = e
f = lambda x: (1 + 1 / x) ** x
for x in [10, 100, 1000, 10000, 100000]:
    print(f'x = {x:<8} f(x) = {f(x):.10f}')
print(f'{"e":>19} = {math.e:.10f}')
```

x = 10	f(x) = 2.5937424601
x = 100	f(x) = 2.7048138294
x = 1000	f(x) = 2.7169239322
x = 10000	f(x) = 2.7181459268

```
x = 100000    f(x) = 2.7182682372
              e = 2.7182818285
```

Limit of a Sequence

For a sequence $a_1, a_2, a_3, \dots$ the notation:

$$\lim_{n \rightarrow \infty} a_n$$

expresses the limiting value of the sequence. If the limit equals L , the sequence is said to converge to L .

```
[7]: # a_n = 1/n converges to 0
a = lambda n: 1 / n
for n in [1, 10, 100, 1000, 10000, 100000]:
    print(f'n = {n:<8}  a_n = {a(n):.10f}')
```

```
n = 1          a_n = 1.0000000000
n = 10         a_n = 0.1000000000
n = 100        a_n = 0.0100000000
n = 1000       a_n = 0.0010000000
n = 10000      a_n = 0.0001000000
n = 100000     a_n = 0.0000100000
```

Limit Superior and Limit Inferior

For a sequence $a_1, a_2, a_3, \dots$ the limit superior and limit inferior are defined as:

$$\limsup_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} [\sup\{a_k : k \geq n\}]$$

$$\liminf_{n \rightarrow \infty} a_n = \lim_{n \rightarrow \infty} [\inf\{a_k : k \geq n\}]$$

These may also be written as $\overline{\lim}$ for lim sup and $\underline{\lim}$ for lim inf. If the sequence converges, then $\limsup = \liminf = \lim$.

```
[8]: # a_n = (-1)^n * (1 + 1/n) oscillates: lim sup = 1, lim inf = -1
a = [(-1)**n * (1 + 1 / n) for n in range(1, 20001)]
for n in [10, 100, 1000, 10000]:
    tail = a[n - 1:]
    print(f'n = {n:<6}  sup = {max(tail):+.6f}  inf = {min(tail):+.6f}')
```

```
n = 10      sup = +1.100000  inf = -1.090909
n = 100     sup = +1.010000  inf = -1.009901
n = 1000    sup = +1.001000  inf = -1.000999
n = 10000   sup = +1.000100  inf = -1.000100
```

Derivatives (Single Variable)

Notation for derivatives of functions of a single independent variable. Covers scalar-valued and vector-valued functions.

Derivative Notation

Given a function f , two standard notations exist for the derivative of f .

Newton's notation uses a prime mark:

$$f'$$

Leibniz's notation uses the differential operator:

$$\frac{df}{dx}$$

Some authors use a roman (upright) d in Leibniz's notation, as in $\frac{df}{dx}$, to distinguish the differential operator from the variable d . Both notations express the rate of change of f with respect to x .

```
[9]: # f(x) = x^2, so f'(x) = df/dx = 2x
f = lambda x: x ** 2
x = 3.0
print(f"f({x}) = {f(x)}")
print(f"f'({x}) = {deriv(f, x):.6f}")
print(f"exact = {2 * x}")
```

```
f(3.0) = 9.0
f'(3.0) = 6.000000
exact = 6.0
```

Higher Order Derivatives

Higher order derivatives in Newton's notation are expressed with additional prime marks. The second derivative is f'' , the third is f''' , and so forth. Sometimes lower case roman numerals replace multiple prime marks, e.g. $f^{(iv)}$. For a positive integer n , the n -th derivative may be written $f^{(n)}$.

In Leibniz's notation, higher order derivatives are expressed with exponents:

$$\frac{df}{dx}, \quad \frac{d^2f}{dx^2}, \quad \frac{d^3f}{dx^3}, \quad \dots$$

The n -th derivative is written $\frac{d^n f}{dx^n}$.

```
[10]: # f(x) = x^4: f' = 4x^3, f'' = 12x^2, f''' = 24x
f = lambda x: x ** 4
```

```

x, h = 2.0, 1e-3
f_1 = (f(x + h) - f(x - h)) / (2 * h)
f_2 = (f(x + h) - 2*f(x) + f(x - h)) / h**2
f_3 = (f(x+2*h) - 2*f(x+h) + 2*f(x-h) - f(x-2*h)) / (2*h**3)
print(f"f'({x}) = {f_1:.4f} (exact: {4*x**3})")
print(f"f''({x}) = {f_2:.4f} (exact: {12*x**2})")
print(f"f'''({x}) = {f_3:.4f} (exact: {24*x})")

```

```

f'(2.0) = 32.0000 (exact: 32.0)
f''(2.0) = 48.0000 (exact: 48.0)
f'''(2.0) = 48.0000 (exact: 48.0)

```

Evaluation at a Point

The value of a derivative at a specific point $x = a$ is written as $f'(a)$ in Newton's notation. In Leibniz's notation, the evaluation bar is used:

$$\left. \frac{df}{dx} \right|_{x=a}$$

If $f(x) = e^x$, then $f'(x) = e^x$ and:

$$\left. \frac{df}{dx} \right|_{x=0} = e^0 = 1$$

```

[11]: # f(x) = e^x, f'(a) = e^a
f = lambda x: math.exp(x)
for a in [0, 1, 2]:
    print(f"df/dx|_(x={a}) = {deriv(f, a):.10f} (exact: {math.exp(a):.10f})")

```

```

df/dx|_(x=0) = 0.9999999995 (exact: 1.0000000000)
df/dx|_(x=1) = 2.7182818285 (exact: 2.7182818285)
df/dx|_(x=2) = 7.3890560914 (exact: 7.3890560989)

```

Dot Notation

When the independent variable is time t , a dot above the function denotes the derivative. If y is a function of t , then $\dot{y}$ denotes dy/dt . Likewise $\ddot{y}$ is the second derivative d^2y/dt^2 :

$$\dot{y} = \frac{dy}{dt}, \quad \ddot{y} = \frac{d^2y}{dt^2}$$

This notation is standard in physics and engineering.

```

[12]: # position y(t) = 5t^2, velocity y_dot = 10t, acceleration y_ddot = 10
y = lambda t: 5 * t ** 2
t, h = 3.0, 1e-4

```

```

y_dot = (y(t + h) - y(t - h)) / (2 * h)
y_ddot = (y(t + h) - 2*y(t) + y(t - h)) / h**2
print(f'y({t})      = {y(t)}')
print(f'y_dot({t}) = {y_dot:.6f}    (exact: {10 * t})')
print(f'y_ddot     = {y_ddot:.4f}    (exact: 10)')

```

```

y(3.0)      = 45.0
y_dot(3.0)  = 30.000000    (exact: 30.0)
y_ddot     = 10.0000    (exact: 10)

```

D Operator Notation

The notation $\frac{d}{dx}$ can be viewed as an operator applied to a function. It is also denoted with a capital D :

$$\frac{d}{dx} [x^2 - 3x + 2] = D [x^2 - 3x + 2] = 2x - 3$$

Higher order derivatives are denoted $\frac{d^n}{dx^n}$ or D^n .

```

[13]: # D[x^2 - 3x + 2] = 2x - 3
f = lambda x: x**2 - 3*x + 2
x = 5.0
print(f'D[x^2 - 3x + 2] at x = {x}: {deriv(f, x):.6f}    (exact: {2*x - 3})')

```

```

D[x^2 - 3x + 2] at x = 5.0: 7.000000    (exact: 7.0)

```

Vector-Valued Derivative

The derivative of a vector-valued function of a single variable is defined component-wise. For $f: \mathbb{R} \rightarrow \mathbb{R}^n$, the derivative is denoted either f' or df/dt . Higher order derivatives are f'' and d^2f/dt^2 .

$$f(t) = \begin{bmatrix} f_1(t) \\ f_2(t) \\ \vdots \\ f_n(t) \end{bmatrix}, \quad f'(t) = \begin{bmatrix} f'_1(t) \\ f'_2(t) \\ \vdots \\ f'_n(t) \end{bmatrix}$$

```

[14]: # f(t) = (cos t, sin t), f'(t) = (-sin t, cos t)
f1 = lambda t: math.cos(t)
f2 = lambda t: math.sin(t)
t = math.pi / 4
f_prime = [deriv(f1, t), deriv(f2, t)]
exact = [-math.sin(t), math.cos(t)]
print(f"f'(pi/4) = [{f_prime[0]:.8f}, {f_prime[1]:.8f}]")
print(f"exact    = [{exact[0]:.8f}, {exact[1]:.8f}]")

```



```
f'(pi/4) = [-0.70710678, 0.70710678]
exact    = [-0.70710678, 0.70710678]
```

Partial Derivatives (Scalar-Valued)

When a function depends on two or more variables, the notion of partial derivative comes into play. This section covers partial derivatives of scalar-valued functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

Partial Derivatives

For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the partial derivative of f with respect to its i -th argument is denoted:

$$\frac{\partial f}{\partial x_i}$$

For a function of two variables x and y , the partial derivatives can also be written as $f_x = \partial_x f = \partial f / \partial x$ and $f_y = \partial_y f = \partial f / \partial y$. Another notation is $\partial_i f$.

The notation D_x means to take the derivative in the x -direction:

$$D_x f = \partial_x f = f_x = \frac{\partial f}{\partial x}$$

```
[15]: # f(x, y) = x^2 * y + y^3
      # df/dx = 2xy,   df/dy = x^2 + 3y^2
      f = lambda x, y: x**2 * y + y**3
      x, y = 2.0, 3.0
      df_dx = partial_deriv(f, [x, y], 0)
      df_dy = partial_deriv(f, [x, y], 1)
      print(f'df/dx at ({x},{y}) = {df_dx:.6f}    (exact: {2*x*y})')
      print(f'df/dy at ({x},{y}) = {df_dy:.6f}    (exact: {x**2 + 3*y**2})')
```

```
df/dx at (2.0,3.0) = 12.000000    (exact: 12.0)
df/dy at (2.0,3.0) = 31.000000    (exact: 31.0)
```

Directional Derivative

If $\mathbf{v}$ is a unit vector, then $D_{\mathbf{v}}f$ is the derivative of f in the $\mathbf{v}$ direction. This is the directional derivative, computed as the dot product of the gradient with the direction vector:

$$D_{\mathbf{v}}f = \nabla f \cdot \mathbf{v}$$

(See: Gradient)

```
[16]: # f(x, y) = x^2 + y^2,   grad f = (2x, 2y)
      # directional derivative along v = (1/sqrt(2), 1/sqrt(2))
      f = lambda x, y: x**2 + y**2
```

```

x, y = 1.0, 1.0
grad = [partial_deriv(f, [x, y], 0), partial_deriv(f, [x, y], 1)]
v = [1 / math.sqrt(2), 1 / math.sqrt(2)]
D_v = sum(g * vi for g, vi in zip(grad, v))
print(f'grad f = [{grad[0]:.4f}, {grad[1]:.4f}]')
print(f'D_v f = {D_v:.6f}    (exact: {2 * math.sqrt(2):.6f})')

```

```

grad f = [2.0000, 2.0000]
D_v f = 2.828427    (exact: 2.828427)

```

Higher Order Partial Derivatives

Higher order partial derivatives are denoted:

$$\frac{\partial^2 f}{\partial x_i \partial x_j}$$

which means to first take the partial derivative of f with respect to x_j and then the partial derivative of that result with respect to x_i :

$$\frac{\partial^2 f}{\partial x_i \partial x_j} = \frac{\partial}{\partial x_i} \left(\frac{\partial f}{\partial x_j} \right)$$

For a function of two variables x and y , the four second-order partials are written f_{xx} , f_{xy} , f_{yx} , f_{yy} . For most functions encountered in practice, the order of differentiation does not matter ($f_{xy} = f_{yx}$).

```

[17]: # f(x, y) = x^2 * y^3
# f_xx = 2y^3, f_yy = 6x^2*y, f_xy = 6xy^2
f = lambda x, y: x**2 * y**3
x, y = 2.0, 3.0
h = 1e-4
f_xx = (f(x + h, y) - 2 * f(x, y) + f(x - h, y)) / h**2
f_yy = (f(x, y + h) - 2 * f(x, y) + f(x, y - h)) / h**2
f_xy = (f(x+h, y+h) - f(x+h, y-h) - f(x-h, y+h) + f(x-h, y-h)) / (4 * h**2)
print(f'f_xx = {f_xx:.4f}    (exact: {2 * y**3})')
print(f'f_yy = {f_yy:.4f}    (exact: {6 * x**2 * y})')
print(f'f_xy = {f_xy:.4f}    (exact: {6 * x * y**2})')

```

```

f_xx = 54.0000    (exact: 54.0)
f_yy = 72.0000    (exact: 72.0)
f_xy = 108.0000   (exact: 108.0)

```

Gradient

The gradient of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the vector of partial derivatives. It is denoted $\text{grad } f$ or ∇f :

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

For a function of three variables, the gradient can be expressed in **i, j, k** notation:

$$\text{grad } f = \nabla f = f_x \mathbf{i} + f_y \mathbf{j} + f_z \mathbf{k}$$

```
[18]: # f(x, y, z) = x^2 + 2yz + z^3
# grad f = (2x, 2z, 2y + 3z^2)
f = lambda x, y, z: x**2 + 2*y*z + z**3
p = [1.0, 2.0, 3.0]
grad = [partial_deriv(f, p, i) for i in range(3)]
exact = [2*p[0], 2*p[2], 2*p[1] + 3*p[2]**2]
print(f'grad f at {p}:')
print(f' numerical = [{grad[0]:.4f}, {grad[1]:.4f}, {grad[2]:.4f}]')
print(f' exact      = {exact}')
```

```
grad f at [1.0, 2.0, 3.0]:
 numerical = [2.0000, 6.0000, 31.0000]
 exact      = [2.0, 6.0, 31.0]
```

Laplacian

The Laplacian operator is denoted with a capital delta Δ . For $f : \mathbb{R}^n \rightarrow \mathbb{R}$:

$$\Delta f = \sum_{i=1}^n \frac{\partial^2 f}{\partial x_i^2}$$

For a function of three variables:

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}$$

The Laplacian can also be written as $\nabla \cdot (\nabla f) = \nabla^2 f$.

```
[19]: # f(x, y, z) = x^2 + y^2 + z^2
# f_xx = 2, f_yy = 2, f_zz = 2, so Laplacian = 6
f = lambda x, y, z: x**2 + y**2 + z**2
p = [1.0, 2.0, 3.0]
h = 1e-4
laplacian = 0
```

```

for i in range(3):
    p_fwd, p_bwd = list(p), list(p)
    p_fwd[i] += h
    p_bwd[i] -= h
    laplacian += (f(*p_fwd) - 2 * f(*p) + f(*p_bwd)) / h**2
print(f'Laplacian at {p} = {laplacian:.6f}    (exact: 6)')

```

Laplacian at [1.0, 2.0, 3.0] = 6.000000 (exact: 6)

Hessian

For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the Hessian matrix Hf is the $n \times n$ matrix of all partial second derivatives:

$$Hf = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

For more information on matrix notation, see the [Linear Algebra notebook](#).

(See: Partial Derivatives)

```

[20]: # f(x, y) = x^3 + x*y^2
      # H = [[6x, 2y], [2y, 2x]]
      f = lambda x, y: x**3 + x * y**2
      x, y = 2.0, 3.0
      h = 1e-4
      f_xx = (f(x+h, y) - 2*f(x, y) + f(x-h, y)) / h**2
      f_yy = (f(x, y+h) - 2*f(x, y) + f(x, y-h)) / h**2
      f_xy = (f(x+h,y+h) - f(x+h,y-h) - f(x-h,y+h) + f(x-h,y-h)) / (4*h**2)
      print(f'Hessian at ({x}, {y}):')
      print(f'    [{f_xx:.2f}, {f_xy:.2f}],')
      print(f'    [{f_xy:.2f}, {f_yy:.2f}]')
      print(f'exact: [{6*x}, {2*y}], [{2*y}, {2*x}]')

```

```

Hessian at (2.0, 3.0):
  [[12.00, 6.00],
   [6.00, 4.00]]
exact: [[12.0, 6.0], [6.0, 4.0]]

```

Derivatives (Vector-Valued)

Derivatives of functions $\mathbf{f}: \mathbb{R}^m \rightarrow \mathbb{R}^n$ that map vectors to vectors. The key objects are the Jacobian matrix, curl, and divergence.

Jacobian

The matrix of all partial derivatives of a vector-valued function $\mathbf{f}: \mathbb{R}^m \rightarrow \mathbb{R}^n$ is called the Jacobian matrix, denoted $D\mathbf{f}$:

$$D\mathbf{f} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_m} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_m} \end{bmatrix}$$

If $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^n$, then $D\mathbf{f}$ is a square matrix and its determinant is also called the Jacobian of $\mathbf{f}$. For a change of coordinates $\mathbf{f}(x, y, z) = (u, v, w)$, the Jacobian determinant is written:

$$\left| \frac{\partial(u, v, w)}{\partial(x, y, z)} \right|$$

For more information on matrix notation, see the [Linear Algebra notebook](#).

```
[21]: # f: R^2 -> R^2, f(x, y) = (x*y, x^2 + y)
# Jacobian: [[y, x], [2x, 1]]
f1 = lambda x, y: x * y
f2 = lambda x, y: x**2 + y
x, y = 3.0, 2.0

J = []
for fi in [f1, f2]:
    row = []
    for j in range(2):
        row.append(partial_deriv(fi, [x, y], j))
    J.append(row)

print('Jacobian at (3, 2):')
for row in J:
    print(f' [{row[0]:8.4f}, {row[1]:8.4f}]')
det_J = J[0][0] * J[1][1] - J[0][1] * J[1][0]
print(f'det(J) = {det_J:.4f} (exact: {y - 2*x**2})')
```

```
Jacobian at (3, 2):
[ 2.0000,  3.0000]
[ 6.0000,  1.0000]
det(J) = -16.0000 (exact: -16.0)
```

Curl

For a function $\mathbf{f}: \mathbb{R}^3 \rightarrow \mathbb{R}^3$, the curl is denoted $\text{curl } \mathbf{f}$ or $\nabla \times \mathbf{f}$:

$$\text{curl } \mathbf{f} = \nabla \times \mathbf{f} = \begin{bmatrix} f_{3,2} - f_{2,3} \\ f_{1,3} - f_{3,1} \\ f_{2,1} - f_{1,2} \end{bmatrix}$$

where $f_{i,j} = \frac{\partial f_i}{\partial x_j}$. The notation $\nabla \times \mathbf{f}$ is inspired by the cross product, treating ∇ as the operator vector:

$$\nabla \times \mathbf{f} = \begin{bmatrix} \partial/\partial x_1 \\ \partial/\partial x_2 \\ \partial/\partial x_3 \end{bmatrix} \times \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix}$$

(See: Gradient)

```
[22]: # f(x, y, z) = (y*z, x*z, x*y)
# curl f = (x - x, y - y, z - z) = (0, 0, 0)
f1 = lambda x, y, z: y * z
f2 = lambda x, y, z: x * z
f3 = lambda x, y, z: x * y
p = [1.0, 2.0, 3.0]
curl = [
    partial_deriv(f3, p, 1) - partial_deriv(f2, p, 2),
    partial_deriv(f1, p, 2) - partial_deriv(f3, p, 0),
    partial_deriv(f2, p, 0) - partial_deriv(f1, p, 1)
]
print(f'curl f at {p} = [{curl[0]:.6f}, {curl[1]:.6f}, {curl[2]:.6f}])
```

```
curl f at [1.0, 2.0, 3.0] = [0.000000, -0.000000, 0.000000]
```

Divergence

For a function $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^n$, the divergence is denoted $\text{div } \mathbf{f}$ or $\nabla \cdot \mathbf{f}$:

$$\text{div } \mathbf{f} = \nabla \cdot \mathbf{f} = \sum_{j=1}^n \frac{\partial f_j}{\partial x_j}$$

The notation is inspired by the dot product, treating ∇ as the operator vector:

$$\nabla \cdot \mathbf{f} = \begin{bmatrix} \partial/\partial x_1 \\ \partial/\partial x_2 \\ \partial/\partial x_3 \end{bmatrix} \cdot \begin{bmatrix} f_1 \\ f_2 \\ f_3 \end{bmatrix}$$

For a scalar-valued function f , the Laplacian (see: Laplacian) can be expressed as $\nabla \cdot (\nabla f) = \nabla^2 f = \Delta f$.

(See: Gradient)

```
[23]: # f(x, y, z) = (x^2, y^2, z^2),  div f = 2x + 2y + 2z
f1 = lambda x, y, z: x**2
f2 = lambda x, y, z: y**2
f3 = lambda x, y, z: z**2
p = [1.0, 2.0, 3.0]
div = sum(partial_deriv(fi, p, i) for i, fi in enumerate([f1, f2, f3]))
print(f'div f at {p} = {div:.6f}    (exact: {2 * sum(p)})')
```

div f at [1.0, 2.0, 3.0] = 12.000000 (exact: 12.0)

Integration

Notation for indefinite and definite integrals, multiple integrals, integration over domains, and line and contour integrals.

Indefinite Integral

Let $f : \mathbb{R} \rightarrow \mathbb{R}$. The notation $\int f(x) dx$ is the indefinite integral of f ; it is a function F whose derivative is f . The function F is called an antiderivative or a primitive of f :

$$\int f(x) dx = F(x) + C \quad \text{where} \quad F'(x) = f(x)$$

Some authors use a roman d in the integral, as in $\int f(x) dx$.

```
[24]: # F(x) = x^3 / 3 is the antiderivative of f(x) = x^2
# verify: F'(x) = x^2
F = lambda x: x**3 / 3
f = lambda x: x**2
x = 5.0
print(f'F(x) = x^3/3,  F({x}) = {F(x):.4f}')
```

F(x) = x³/3, F(5.0) = 41.6667

F'(5.0) = 25.000000

f(5.0) = 25.0

Definite Integral

The definite integral of f from a to b is denoted:

$$\int_a^b f(x) dx$$

Its value is given by the Fundamental Theorem of Calculus:

$$\int_a^b f(x) dx = F(x) \Big|_a^b = F(b) - F(a)$$

where F is any antiderivative of f . The left and right endpoints may be $-\infty$ and ∞ , respectively.

```
[25]: # numerical integration via midpoint Riemann sum
def integrate(f, a, b, n=100000):
    dx = (b - a) / n
    return sum(f(a + (i + 0.5) * dx) for i in range(n)) * dx

# integral of x^2 from 0 to 3 = [x^3/3]_0^3 = 9
result = integrate(lambda x: x**2, 0, 3)
print(f'integral of x^2 from 0 to 3 = {result:.8f}    (exact: 9.0)')
```

```
integral of x^2 from 0 to 3 = 9.00000000    (exact: 9.0)
```

Multiple Integrals

Multiple integrals extend integration to functions of several variables. A double integral is written:

$$\iint f(x, y) dx dy$$

This means to integrate $f(x, y)$ first with respect to x (holding y constant), then integrate the result with respect to y . It is equivalent to the iterated integral:

$$\int \left[\int f(x, y) dx \right] dy$$

```
[26]: # double integral of x*y over [0,1] x [0,1] = 1/4
n = 300
dx = dy = 1.0 / n
total = 0
for i in range(n):
    x = (i + 0.5) * dx
    for j in range(n):
        y = (j + 0.5) * dy
        total += x * y * dx * dy
print(f'integral of x*y over [0,1]^2 = {total:.8f}    (exact: {1/4})')
```

```
integral of x*y over [0,1]^2 = 0.25000000    (exact: 0.25)
```

Domain Subscripts

Subscripts on integrals indicate the domain of integration. The typical notation is $\int_A f ds$ where A is a set. Some examples:

$$\int_{[0,1]} x^2 dx \quad \text{means} \quad \int_0^1 x^2 dx$$

$$\int_{\mathbb{R}} \exp\{-x^2\} dx \quad \text{means} \quad \int_{-\infty}^{\infty} e^{-x^2} dx$$

$$\int_{[0,1]^2} (x-y)^2 dx dy \quad \text{means} \quad \int_0^1 \int_0^1 (x-y)^2 dx dy$$

```
[27]: # integral of exp(-x^2) over R = sqrt(pi)
      # approximate by integrating over [-10, 10]
      f = lambda x: math.exp(-x**2)
      result = integrate(f, -10, 10)
      print(f'integral of exp(-x^2) over R = {result:.8f}')
      print(f'sqrt(pi)                        = {math.sqrt(math.pi):.8f}')
```

```
integral of exp(-x^2) over R = 1.77245385
sqrt(pi)                    = 1.77245385
```

Line Integral and Contour Integral

When the subscript on the integral is a curve γ :

$$\int_{\gamma} f ds$$

this denotes the line integral of f along the curve γ . If the curve is closed (it begins and ends at the same point), the contour integral notation is used:

$$\oint_{\gamma} f ds$$

When the contour is the boundary of a domain D , the notation may be:

$$\int_{\partial D} f ds \quad \text{or} \quad \oint_{\partial D} f ds$$

where ∂D indicates the boundary of D .

```
[28]: # line integral of  $f(x,y) = x^2 + y^2$  along the unit circle
# parametrize:  $x = \cos(t)$ ,  $y = \sin(t)$ ,  $ds = dt$ 
# integral =  $\int_0^{2\pi} (\cos^2 t + \sin^2 t) dt = 2\pi$ 
n = 100000
dt = 2 * math.pi / n
total = sum(
    (math.cos((i+0.5)*dt)**2 + math.sin((i+0.5)*dt)**2) * dt
    for i in range(n)
)
print(f'line integral of  $x^2+y^2$  along unit circle = {total:.8f}')
print(f'exact ( $2\pi$ ) = {2 * math.pi:.8f}')
```

line integral of x^2+y^2 along unit circle = 6.28318531
 exact (2π) = 6.28318531

Convolution and Transforms

Convolution combines two functions into a new function. Integral transforms (Laplace, Fourier) map functions between domains.

Convolution

If f and g are functions from $\mathbb{R}$ to $\mathbb{R}$, their convolution $f * g$ is a new function defined by:

$$(f * g)(x) = \int_{-\infty}^{\infty} f(t) g(x - t) dt$$

The convolution extends to discrete sequences. Given sequences $a = (a_0, a_1, a_2, \dots)$ and $b = (b_0, b_1, b_2, \dots)$, their convolution $c = a * b$ is:

$$c_n = \sum_{k=0}^n a_k b_{n-k}$$

```
[29]: # discrete convolution of  $a = [1, 2, 3]$  and  $b = [4, 5, 6]$ 
a = [1, 2, 3]
b = [4, 5, 6]
n_out = len(a) + len(b) - 1
c = []
for n in range(n_out):
    c_n = 0
    for k in range(n + 1):
        if k < len(a) and 0 <= n - k < len(b):
            c_n += a[k] * b[n - k]
    c.append(c_n)
print(f'a = {a}')
print(f'b = {b}')
```

```
print(f'a * b = {c}')
```

```
a = [1, 2, 3]
b = [4, 5, 6]
a * b = [4, 13, 28, 27, 18]
```

Laplace Transform

Let $f : \mathbb{R}_+ \rightarrow \mathbb{R}$. The Laplace transform of f is denoted $\mathcal{L}f$. It is a new function defined on the nonnegative real numbers:

$$(\mathcal{L}f)(s) = \int_0^\infty f(t) e^{-st} dt$$

The inverse Laplace transform is denoted $\mathcal{L}^{-1}$.

```
[30]: # Laplace transform of f(t) = e^{-t} is 1/(s+1)
def laplace(f, s, upper=50, n=50000):
    dt = upper / n
    return sum(f((i+0.5)*dt) * math.exp(-s*(i+0.5)*dt) * dt for i in range(n))

f = lambda t: math.exp(-t)
for s in [0.5, 1.0, 2.0, 5.0]:
    Lf = laplace(f, s)
    print(f'(Lf)({s}) = {Lf:.8f}    exact: 1/(s+1) = {1/(s+1):.8f}')
```

```
(Lf)(0.5) = 0.66666660    exact: 1/(s+1) = 0.66666667
(Lf)(1.0) = 0.49999992    exact: 1/(s+1) = 0.50000000
(Lf)(2.0) = 0.33333321    exact: 1/(s+1) = 0.33333333
(Lf)(5.0) = 0.16666642    exact: 1/(s+1) = 0.16666667
```

Fourier Transform

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ (or more generally, $f : \mathbb{R} \rightarrow \mathbb{C}$). The Fourier transform of f is denoted $\mathcal{F}f$. It is a new function $F : \mathbb{R} \rightarrow \mathbb{C}$:

$$(\mathcal{F}f)(\omega) = F(\omega) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} f(x) \exp[-i\omega x] dx$$

Variations on this definition differ in the coefficient outside the integral and/or the sign in the exponential term. The inverse Fourier transform is denoted $\mathcal{F}^{-1}$.

```
[31]: # Fourier transform of f(x) = exp(-x^2/2) is F(w) = exp(-w^2/2)
def fourier(f, omega, lo=-10, hi=10, n=50000):
    dx = (hi - lo) / n
    total = sum(f(lo+(i+0.5)*dx) * cmath.exp(-1j*omega*(lo+(i+0.5)*dx)) * dx
    for i in range(n))
```

```

    return total / math.sqrt(2 * math.pi)

f = lambda x: math.exp(-x**2 / 2)
for omega in [0.0, 0.5, 1.0, 2.0]:
    F_w = fourier(f, omega)
    print(f'F({omega}) = {F_w.real:.8f}    exact = {math.exp(-omega**2/2):.8f}')

F(0.0) = 1.00000000    exact = 1.00000000
F(0.5) = 0.88249690    exact = 0.88249690
F(1.0) = 0.60653066    exact = 0.60653066
F(2.0) = 0.13533528    exact = 0.13533528

```

Discrete Fourier Transform

The discrete Fourier transform applies to a finite sequence $a = (a_0, a_1, \dots, a_{n-1})$ and produces a new sequence $A = (A_0, A_1, \dots, A_{n-1})$:

$$A_k = \sum_{j=0}^{n-1} a_j e^{-j \cdot 2\pi i \cdot k / n}$$

Some authors scale this by a factor of $1/\sqrt{n}$. Discrete Fourier transforms can be efficiently computed using the fast Fourier transform (FFT) algorithm. The discrete Fourier transform is closely related to the discrete cosine transform (DCT).

For more information on complex numbers, see the Numbers notebook.

```

[32]: # DFT of a = [1, 2, 3, 4]
a = [1, 2, 3, 4]
n = len(a)
A = []
for k in range(n):
    A_k = sum(a[j] * cmath.exp(-2j * cmath.pi * j * k / n) for j in range(n))
    A.append(A_k)
print(f'a = {a}')
print('DFT(a):')
for k in range(n):
    print(f'  A[{k}] = {A[k].real:+8.4f} {A[k].imag:+8.4f}j')

```

```

a = [1, 2, 3, 4]
DFT(a):
  A[0] = +10.0000 +0.0000j
  A[1] = -2.0000 +2.0000j
  A[2] = -2.0000 -0.0000j
  A[3] = -2.0000 -2.0000j

```

Fourier Series

The Fourier series of a periodic function $f : [-\pi, \pi] \rightarrow \mathbb{R}$ represents f as a sum of sine and cosine terms:

$$f(t) = a_0 + \sum_{k=1}^{\infty} (a_k \cos kt + b_k \sin kt)$$

This can be written in exponential form:

$$f(t) = \sum_{k=-\infty}^{\infty} \hat{f}(k) e^{ikt}$$

The numbers $\hat{f}(k)$ are the Fourier coefficients, computed as:

$$a_0 = \frac{1}{2\pi} \int_{-\pi}^{\pi} f(t) dt, \quad a_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(t) \cos(kt) dt, \quad b_k = \frac{1}{\pi} \int_{-\pi}^{\pi} f(t) \sin(kt) dt$$

(See: Definite Integral)

```
[33]: # Fourier coefficients of f(t) = t on [-pi, pi]
# a_0 = 0, a_k = 0, b_k = 2*(-1)^(k+1) / k
n_pts = 50000
dt = 2 * math.pi / n_pts
print('Fourier coefficients of f(t) = t:\n')
for k in range(0, 5):
    a_k = sum((-math.pi+(i+0.5)*dt) * math.cos(k*(-math.pi+(i+0.5)*dt)) for i in range(n_pts)) * dt / math.pi
    if k == 0:
        a_k /= 2
        print(f'a_0 = {a_k:+.8f} (exact: 0)')
    else:
        b_k = sum((-math.pi+(i+0.5)*dt) * math.sin(k*(-math.pi+(i+0.5)*dt)) for i in range(n_pts)) * dt / math.pi
        exact_b = 2 * (-1)**(k+1) / k
        print(f'a_{k} = {a_k:+.8f} b_{k} = {b_k:+.8f} (exact b_{k} = {exact_b:+.8f})')
```

Fourier coefficients of $f(t) = t$:

```
a_0 = -0.00000000 (exact: 0)
a_1 = +0.00000000 b_1 = +2.00000000 (exact b_1 = +2.00000000)
a_2 = -0.00000000 b_2 = -1.00000000 (exact b_2 = -1.00000000)
a_3 = +0.00000000 b_3 = +0.66666667 (exact b_3 = +0.66666667)
a_4 = -0.00000000 b_4 = -0.50000001 (exact b_4 = -0.50000000)
```